Chunking in Retrieval-Augmented Generation

Retrieval-augmented generation answers a question in two steps. First it finds passages that look relevant, then it asks a language model to answer from those passages alone. The passages are chunks: pieces of the original documents, cut before anything is indexed. How they are cut decides what the retriever can ever find.

Why chunk boundaries matter

A retriever scores each chunk as a whole. When a boundary falls in the middle of an explanation, the question lands on one half and the answer on the other, and neither half scores well on its own. The model then receives a passage that mentions the topic but never resolves it.

Boundaries also decide what travels together. A definition separated from the term it defines, or a table separated from its caption, loses the context that made it useful. The embedding of such a chunk describes a fragment, not an idea, so it drifts away from the questions it should match.

A useful rule of thumb: a chunk should answer one question well. If you cannot say in a sentence what a chunk is about, a retriever will struggle to decide when it is relevant, and a reader will struggle to use it.

Bad boundaries are hard to see from the final answer. The model is fluent either way, so the symptom is usually an answer that is vague or quietly wrong. Looking at the chunks themselves, before indexing, is the quickest way to catch the problem.

Chunking is usually done once, when documents are added, and then forgotten. That is a pity, because it is one of the few settings that changes what every later step can do.

The examples in this primer use English prose, but the same ideas hold for any language. Only the numbers change, because words and tokens have different lengths.

Chunk size and overlap

Chunk size is a trade between precision and context. Small chunks match a question closely, because every sentence in them is about the same thing. Large chunks carry more surrounding context, but their embedding averages over several topics and matches none of them sharply.

There is also a budget on the other side. Every retrieved chunk is pasted into the prompt, so the number of chunks you retrieve times their size must fit in the context window, with room left for the question and the answer. Doubling the chunk size roughly halves how many distinct passages the model can see.

Overlap repeats the end of one chunk at the start of the next. It is a cheap insurance policy against a boundary that lands in the wrong place: a sentence cut at the edge of one chunk appears whole in its neighbour. The cost is a larger index and near-duplicate results, since adjacent chunks now share text.

A common starting point is a few hundred tokens per chunk with an overlap of ten to twenty percent. Treat these numbers as a first guess to measure, not a rule. Documents with short, self-contained entries, such as a glossary or an FAQ, often want much smaller chunks than long narrative reports.

Overlap should always be smaller than the chunk itself. When the two are equal, each step moves forward by nothing, and a splitter either loops or refuses to run.

Small chunks have a second cost. A question that needs two facts may find them in two different chunks, and the retriever has to bring back both for the model to answer.

Large chunks have the opposite problem. A chunk of a thousand tokens may hold the answer in its fifth paragraph, and the model has to find it among everything else.

Overlap does not fix a chunk that is simply too small. It moves a few sentences across a boundary; it cannot give a two-sentence chunk the context of a whole section.

Most splitters let you count size in characters or in tokens. Tokens match what the model sees; characters are simpler to reason about and easier to show on screen.

Structure-aware splitting

A section under one heading usually discusses one subject, which is exactly what a good chunk should do. Fixed-size splitting ignores the document's own shape. Structure-aware splitting uses it: headings, paragraphs, list items and tables become the natural places to cut.

This only works when the parser recovers the structure. A layout-aware parser labels headings, footers and tables, while a plain text extractor returns a flat stream of lines. Cleaning comes next: running headers, footers and page numbers are removed, so they do not end up in the middle of chunks.

A useful rule of thumb: a chunk should answer one question well. If you cannot say in a sentence what a chunk is about, a retriever will struggle to decide when it is relevant, and a reader will struggle to use it.

A structure-aware splitter still needs a size limit. A long section is split again inside itself, preferably at paragraph and then sentence boundaries, and each piece keeps its heading path so that the retriever knows where it came from.

Choosing a starting point

Start simple, look at the chunks, then ask a handful of real questions and check which chunks come back. Change one setting at a time and compare the results side by side. The right configuration is the one that retrieves the passage a careful reader would have picked.

Keep the questions you used. Each time a setting changes, ask them again; a change that helps one question often hurts another, and only a fixed set of questions shows both.

When a question fails, look at where its answer sits in the document before changing anything. The fix for an answer split across two chunks is not the fix for an answer the search never found.

A good test set is small and specific. Ten questions with known answers, each tied to one sentence in the document, show more than a hundred vague ones.

Chunk size in numbers

Take a report of forty pages and about twenty thousand words. Cut into chunks of two hundred tokens, it becomes roughly a hundred and thirty pieces; cut into chunks of eight hundred tokens, about thirty-five.

The smaller cut gives the retriever more targets, each sharper and each saying less. The larger cut gives fewer targets, and each one carries a paragraph or two of context with it.

The three things that decide a good chunk size for a collection are how long the passages are that answer a typical question, how many retrieved passages the prompt can hold once the question, the instructions and room for the answer have been set aside, and how much surrounding text a reader needs before a passage makes sense on its own, which for a reference table is very little and for an argued report can be a whole page.

Token counts and character counts are not the same. English averages about four characters per token, so a chunk of four hundred characters holds roughly a hundred tokens, and a sentence of a hundred words needs about a hundred and thirty.

Measure rather than guess: build the index at two sizes, ask the same ten questions of both, and count how many answers come back in the top five. The difference is usually larger than people expect.

In one test of five collections the best size was different for each, and so were the reasons.

The help-desk articles did best with pieces of about two hundred tokens, because each article answered a single problem in a few short paragraphs.

The legal contracts did best with pieces of about eight hundred tokens, because their clauses refer to each other and lose their meaning when cut apart.

The product questions and answers did best with one entry per piece, whatever its length, because every entry already stood on its own.

The meeting notes did best at about three hundred tokens, cut wherever a new speaker began, because each turn in the discussion made one point.

The research papers did best at about five hundred tokens, cut at the ends of paragraphs, because their arguments ran across several sentences.

What a heading does to a split

A heading is a promise about what follows. A splitter that starts a new chunk at each heading keeps that promise intact, and a chunk that begins with its heading tells the retriever what it is about before the first sentence.

Short sections are the exception. A heading followed by a single sentence makes a chunk too small to answer anything, so splitters usually merge it with the next section until the chunk reaches a useful size.

Long sections are split again inside themselves. Each part keeps the heading of its section, often with the headings above it, so a piece from the middle of a chapter still says which chapter it came from.

Headings help only when the parser finds them. In a scanned report or a plain text export they look like any other line, and a heading-based splitter falls back to cutting by size.

Some documents have headings that say little, such as Introduction or Results. Adding the document’s title to the heading path gives such chunks enough context to be told apart.

Headings also travel into the embedding. When the heading path is added to the start of a chunk, its embedding moves towards the topic the heading names, which helps short chunks most.

The cost is length. A heading path of three levels can add twenty or thirty tokens to every chunk, which matters when chunks are small and the context window is tight.

Numbered headings need care. A splitter that treats every numbered line as a heading will cut a numbered list into pieces of one item each.

In the end, headings are a hint, not a rule. The splitter still checks the size of every chunk and cuts again when a section runs long.

A short glossary

Chunk. A piece of a document, cut before indexing, that the retriever scores as a whole.

Chunk size. How long each piece may be, counted in characters or in tokens.

Overlap. Text repeated at the start of a chunk from the end of the one before it.

Splitter. The step that decides where one chunk ends and the next begins.

Embedding. A list of numbers that stands for what a chunk means, used to compare it with a question.

Index. The store of chunks and their embeddings that a search runs against.

Retriever. The step that scores every chunk against a question and returns the best.

Top k. How many of the best-scoring chunks a search hands on to the next step.

Reranker. A second, slower model that reads the question and each returned chunk together.

Hybrid search. A search that combines keyword matches with embedding matches.

Context window. The amount of text a language model can read at once.

Heading path. The headings above a chunk, kept with it so it says where it came from.

Recall. The share of the passages that answer a question that the search brought back.

Precision. The share of what the search brought back that answers the question.

Token. A small unit of text, often part of a word, that models read and count.

Semantic chunking. Cutting where the meaning changes, found by comparing the embeddings of neighbouring sentences.

Parent and child. Small chunks searched for precision, each pointing to a larger chunk handed to the model for context.

Evidence. The sentence in a document that answers a question, used to check whether a search found it.

Hit rate. The share of test questions whose evidence came back in the pieces checked.